A DECISION FRAMEWORK FOR PRODUCT LEADERS NAVIGATING MARKET UNCERTAINTY

PRODUCT STRATEGY · Q3 2025

From Signal to Strategy

SECTION 01 · FOUNDATIONS

The landscape has shifted. Here is what that means for us.

MODULE 02

Before we score signals, we need to agree on what a signal is.

The word is overloaded. We use it to mean anything from a customer complaint to a macro trend. This framework requires a tighter definition.

CONTEXT · COMPETITIVE DYNAMICS

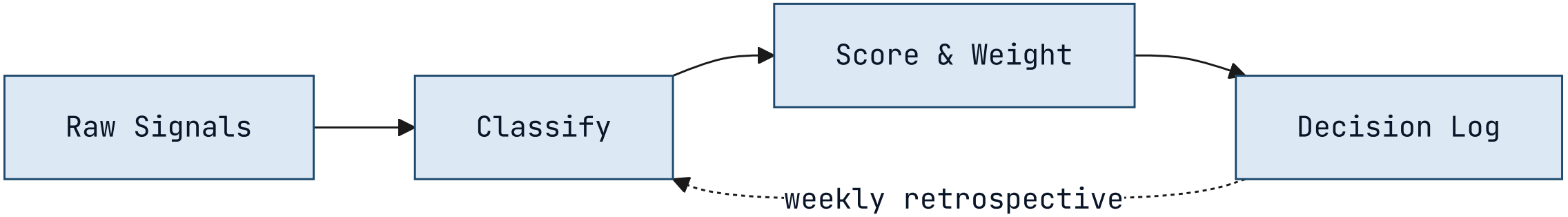
The window for differentiation is narrowing.

Three converging forces — commoditized infrastructure, compressed release cycles, and rising customer switching costs — have reduced the average durable advantage window from 36 months to under 14. Teams that cannot identify signal from noise in that window will consistently miss timing.

ARCHITECTURE · SIGNAL PIPELINE

How signals move from input to decision.

Four-stage processing pipeline — weekly cadence



OVERVIEW · FRAMEWORK DESIGN

Built for speed without sacrificing rigor.

The framework was designed around a hard constraint: product decisions must close in 72 hours or the opportunity cost of delay exceeds the cost of a suboptimal decision. Every component is optimized for that window.

Visual: decision velocity vs. decision quality scatter plot — 72-hour decision window highlighted

IMPACT · PILOT RESULTS

Six months of results across four product teams.

Measured against pre-framework baseline, same teams, same market conditions.

73%	4.2×	18	91%
FASTER CLOSE	SIGNAL RECALL	DECISIONS LOGGED	TEAM ALIGNMENT

The framework has four components.

Signal Intake

Weekly structured collection across customer conversations, market data, and competitive moves. Normalized into a common schema before scoring.

Scoring Model

Each signal scored on three dimensions: confidence, recency, and strategic relevance. Weights are team-configurable and reviewed quarterly.

Decision Log

Every decision recorded with the signals that informed it, the options considered, and the criteria applied. Feeds the calibration loop.

Calibration Loop

Monthly retrospective that compares predicted outcomes to actual outcomes and adjusts scoring weights accordingly.

Code in card headers and body text.

Signal Intake

`v2.4`

Handles 94% of structured signals without manual intervention. Average latency: `4 min` from ingestion to scored entry.

Scoring Model

`configurable`

Three dimensions: confidence, recency, relevance. Default weights are `33 / 33 / 33` — adjust after your first retrospective.

Decision Log

`required`

Every prioritization change above `P2` must carry a logged rationale. No log, no change.

Calibration Loop

`monthly`

Compares predicted outcomes to actuals. First meaningful weight update happens after `2 cycles`.

Signal Intake produces three outputs.

1

Weekly Signal Brief

A ranked list of the top 10 signals from the prior week, with confidence scores and source attribution. Distributed to product leads every Monday morning.

2

Anomaly Alerts

Real-time flags when a signal exceeds the 2σ threshold on any dimension. Routed directly to the accountable PM with a 4-hour response SLA.

3

Monthly Signal Index

The source of truth for the calibration loop. A complete record of all signals logged, scored, and resolved in the prior month. Required reading before each retrospective.

Two failure modes the framework is designed to prevent.

False signal amplification. A single loud voice — one enterprise customer, one analyst report, one competitive announcement — dominates the decision without being weighed against the full signal set. The scoring model prevents any single source from exceeding 30% of the total signal weight in a given decision.

Signal hoarding. Teams collect signals but do not log decisions, so the calibration loop has nothing to learn from. The Decision Log is a required artifact for any prioritization change above P2 severity. No log, no change.

Two intake modes for different signal types.

Structured Intake

Signals with clear schema: NPS verbatims, support ticket categories, feature request volumes, win/loss notes. Ingested automatically via API connectors. Scored on arrival. Zero manual handling.

Unstructured Intake

Signals without schema: field observations, conference conversations, analyst briefings, competitive demos. Require human classification before scoring. Routed to the signal owner for a 48-hour classification window.

Scoring model: before and after the calibration loop.

Before Calibration

Equal weights across all three dimensions. Confidence, recency, and relevance each contribute 33% to the final score. Simple, consistent, but blind to what your market actually rewards.

›

After Calibration

Weights reflect your team's historical signal accuracy. If recency has consistently been the weakest predictor for your product, it gets downweighted. The model becomes a record of what you have learned.

The shift from equal weights to calibrated weights takes two retrospective cycles — roughly 60 days from adoption.

> *The signal was always there. We just didn't have a system that forced us to look at it before we'd already decided.*

— Head of Product, Pilot Team 3

How a decision moves through the framework.

1. **Signal Logged**
   Owner classifies and submits to intake queue
2. **Scored**
   Model applies current weights, generates score
3. **Brief Published**
   Signal appears in weekly brief with rank
4. **Decision Logged**
   PM records rationale, signals, predicted outcome
5. **Retrospective**
   Outcome scored, weights updated accordingly

What the framework does not do.

- It does not make decisions — it structures the information that humans use to decide.
- It does not replace customer discovery — it scores and routes what discovery surfaces.
- It does not work without the Decision Log — calibration requires outcome data to learn from.
- It does not guarantee alignment — it surfaces disagreement earlier, which still requires resolution.
- It does not scale down to individual feature decisions — it is designed for prioritization above P2.

Four things that must be true before you begin.

01 You have a regular prioritization cadence — at minimum monthly.

02 At least one person owns signal collection full-time or as a primary responsibility.

03 Leadership has agreed to log decisions with rationale, not just outcomes.

04 You have 90 minutes per week to run the intake and scoring process.

CALIBRATION RESULT · 6-MONTH PILOT

14X

Return on signal investment — measured as decisions that reached the right outcome on the first attempt, versus the baseline rate before the framework was adopted.

SECTION 02

Scoring Model Deep Dive

Dark panel + content · split-panel

WHAT THIS SECTION COVERS

The scoring model is the most configurable component. This section covers the three dimensions, how weights are set initially, and how calibration updates them over time.

1 **Confidence**

How many independent sources corroborate the signal. Ranges 1–5.

2 **Recency**

Time-decay applied from signal date to scoring date. Half-life is team-configurable.

3 **Strategic Relevance**

Manual score from the signal owner. Ranges 1–5. Requires justification above 4.

WHAT WOULD HELP US MOVE FORWARD

Next step is a working session, not a debate.

Walk these questions with me in 60–90 minutes. The output is either a design we can execute, or a shared list of what needs more work before we commit.

FINDING 01 · STRUCTURED INTAKE

Structured intake performed above expectations — volume and latency were not concerns.

What worked

API connectors handled 94% of structured signals without manual intervention. Average scoring latency was 4 minutes from ingestion. Schema normalization held across all five connected sources.

What required tuning

NPS verbatim classification had an 18% error rate in the first two weeks. Required a training pass on the classification model before accuracy reached the 92% target.

- **Viable as designed — NLP classification requires a 2-week warm-up period on new deployments.**

Key insight works on any card-bearing layout.

Signal Intake

Weekly structured collection across customer conversations, market data, and competitive moves.

Scoring Model

Each signal scored on three dimensions: confidence, recency, and strategic relevance.

Decision Log

Every decision recorded with the signals that informed it and the criteria applied.

Calibration Loop

Monthly retrospective that compares predicted outcomes to actual outcomes.

KEY INSIGHT

The calibration loop is what separates teams that learn from teams that repeat the same mistakes.

[1] Trailing blockquote becomes a key insight. Trailing paragraph becomes a below-note.

Three scoring failure modes found in the pilot.

1 Recency dominance

High-recency noise crowding out durable signal. Teams set recency weight above 50% in the first calibration pass. Corrected by capping recency weight at 40% until two calibration cycles complete.

2 Source concentration

Single-customer signals inflating confidence scores. One enterprise customer's verbatims represented 34% of all structured intake in month one. Corrected by adding a source-diversity floor to the scoring model.

3 Outcome misclassification

PMs logging predicted outcomes that were too vague to score at retrospective. "Improve retention" is not scoreable. "Reduce 30-day churn from 8.2% to below 7%" is.

Four requirements every decision system must meet.

01 Speed

Decisions must close within the window they are relevant to. Systems that add latency consume the value they exist to protect.

02 Auditability

Every prioritization decision above a threshold must carry a traceable rationale. Required for alignment and compliance.

03 Adoption

If the team won't use it weekly, calibration never runs and the model never improves. Ninety minutes per PM is the ceiling.

04 Calibration

The system must improve over time. A static scoring model is a spreadsheet with extra steps.

We evaluated four intake tools against the criteria.

Tool A · Chorus

✓ Speed | ~ Auditability | ✓ Adoption | ✗ Calibration

Strong call recording and summarization. No decision logging or calibration loop. Requires separate tooling for everything downstream of intake.

Tool B · Productboard

✗ Speed | ✓ Auditability | ✓ Adoption | ✗ Calibration

Solid intake and prioritization. Decision logging exists but is manual and rarely used. No calibration mechanism. Setup takes 3–4 weeks.

Tool C · Notion

✓ Speed | ✓ Auditability | ~ Adoption | ✗ Calibration

Flexible enough to build the full system. But building it takes 40+ hours and the result is fragile. Teams abandon maintenance after the first quarter.

Tool D · Sprig + Decision Log

✓ Speed | ✓ Auditability | ✓ Adoption | ✓ Calibration

Meets all four criteria within the 90-minute weekly budget. Reaches production in the same week it is adopted. Recommended.

The four tools side by side.

CRITERION	CHORUS	PRODUCTBOARD	NOTION	SPRIG + LOG
Speed	✓	✗	✓	✓
Auditability	✗	✓	✓	✓
Adoption	✓	✓	✗	✓
Calibration	✗	✗	✗	✓
Setup time	1 day	3–4 weeks	40+ hours	Same day

Evaluated against the same four teams and the same 90-minute weekly budget constraint.

Applying the criteria to the tools — here is where the evidence points.

THE EVIDENCE FAVORS TOOL D

Sprig combined with a lightweight Decision Log meets all four criteria within the 90-minute weekly budget, reaches production in the same week it is adopted, and leaves a clean exit ramp if a better native solution emerges.

The path is not self-executing

Sprig requires a connector built to your NPS and support platforms. Budget 4–6 hours of engineering time in week one. After that, zero maintenance overhead.

The Decision Log is the hardest part

Not technically. Culturally. PMs need to log decisions with predicted outcomes before they close, not after. This is a habit change, not a tool change.

Two options with a connector and an explanatory note below.

Option A · Label

Body text describing the first option. Enough detail to fill the card naturally and show how the layout handles a few lines of prose.

›

Option B · Label

Body text describing the second option. The connector arrow between them implies direction or causality — before/after, input/output, cause/effect.

The below-note sits under the cards after a hairline rule. Use it for a single contextual sentence.

How to roll this out across your organization.

STEP 01

Pick one team and one decision type

Start with a team that already has a regular prioritization rhythm. Apply the framework only to a single decision category for the first 30 days.

STEP 02

Log everything, decide nothing differently

In the first month, do not change how you make decisions. Just log signals and decisions as you would have made them anyway.

STEP 03

Run your first retrospective

At day 30, score the logged decisions against outcomes. This is where the model gets its first calibration pass.

STEP 04

Expand to a second team

With one retrospective complete, you have evidence. Use it to onboard the second team with real data, not promises.

The six signal dimensions, what they measure, and how they are scored.

01	**Confidence**	Number of independent sources corroborating the signal	1–5 · Auto-scored
02	**Recency**	Time-decay from signal date, configurable half-life	0.0–1.0 · Auto-scored
03	**Relevance**	Alignment to current strategic bets, owner-scored	1–5 · Manual
04	**Reach**	Number of customers or segments affected	1–5 · Auto-scored
05	**Effort**	Engineering and design cost to act on the signal	1–5 · Manual
06	**Confidence delta**	Change in confidence score since last scoring cycle	−5 to +5 · Auto

HEADER AND FOOTER

Header stays uppercase — footer renders as written.

Set `header:` and `footer:` in frontmatter for deck-level labels, or use per-slide comment directives. The header uses uppercase text-transform automatically, so you write it in any case. The footer renders exactly as written.

IMPLEMENTATION · TOKEN PIPELINE

The tokenization call is three lines of application code.

JAVASCRIPT · SDK V2 INTERFACE

```javascript
import { TokenVault } from "@company/token-sdk";

const vault = new TokenVault({ keyFile: "./vault.key" });

// Tokenize at ingestion
const token = await vault.tokenize(ssn, { field: "ssn", tenant: "acme" });

// Detokenize only at point of use — every call is logged
const plaintext = await vault.detokenize(token, { requestor: "claims-svc" });
```

BEFORE & AFTER · KEY DISTRIBUTION

File-distributed keys versus vault-integrated keys.

BEFORE · FILE-DISTRIBUTED

```python
# Key material on disk — anyone with
# filesystem access can read it
with open('./vault.key', 'rb') as f:
    key = f.read()

cipher = AES(key)
token = cipher.encrypt(ssn)
```

AFTER · HSM / KMS INTEGRATED

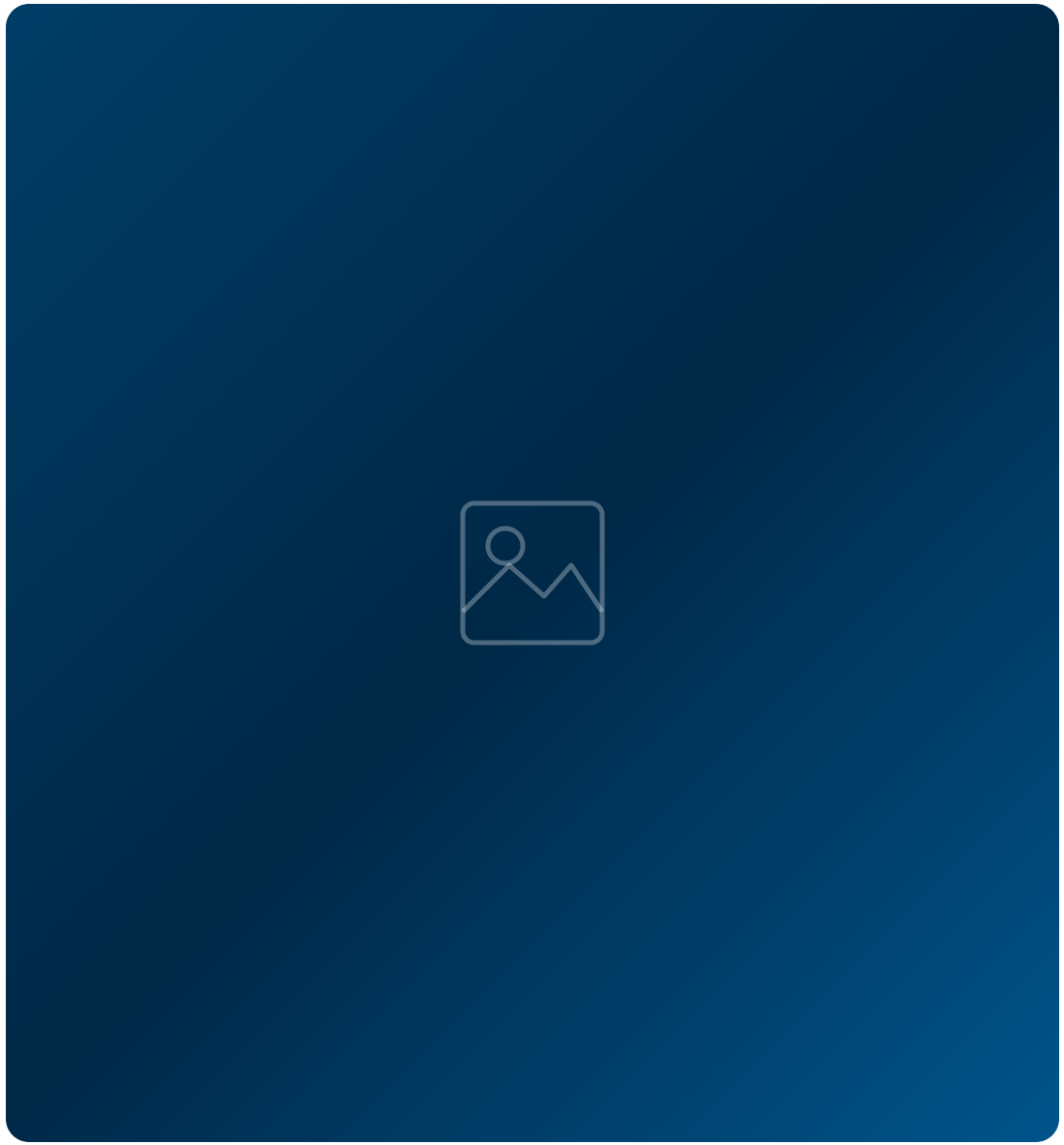
```python
# Key never leaves the HSM —
# every operation is audited
import boto3

kms = boto3.client('kms')
token = kms.encrypt(
    KeyId='alias/tokenization',
    Plaintext=ssn
)['CiphertextBlob']
```

LAYOUT · IMAGE

Images sit naturally beside text when you need visual evidence.

Use `_class: image` with `![bg right](url)` — image-right is the default. Add `left` to flip: `_class: image left`.



LAYOUT · IMAGE LEFT

Lead with the image, follow with the argument.

Add `left` to the class: `_class: image left` with `![bg left](url)`. Only the padding and placeholder position change — everything else is shared.

[DASHBOARD SCREENSHOT · SIGNAL BRIEF — WITH CAPTION]

Weekly Signal Brief — the primary output of the intake pipeline, distributed every Monday

DARK VARIANT · ANY LAYOUT CLASS

The dark modifier works on any layout.

Add *dark* alongside any class — palette remaps automatically

DARK VARIANT · CONTENT

The token system handles dark without per-element overrides.

All colours reference CSS variables — `--bg`, `--text-heading`, `--text-body`, `--border` — that remap when `dark` is added. Cards, headings, body text, and borders all shift automatically. The spectrum bar is suppressed on dark slides.

[DASHBOARD SCREENSHOT · SIGNAL BRIEF — DARK]

Weekly Signal Brief — the primary output of the intake pipeline, distributed every Monday

DARK VARIANT · LIST

The card stack renders cleanly on dark backgrounds.

Every card uses `--bg-alt` for fill and `--border` for the border — both remap in dark mode.

The accent left border uses `--accent` which is unchanged — the gold reads well against dark.

Body text shifts to `--text-body` which in dark mode is a warm light tone, not pure white.

DARK VARIANT · CARDS STACKED

Two-card layouts work equally well inverted to dark.

The architecture introduces a single key distribution question: what protects the file containing key material, and what is the blast radius if it leaves the host? Every other question in this document depends on the answer.

The pattern here is the same as any page of written argument — claim, then support. The dark palette does not change the information density or the reading rhythm.